

Sistemas Tolerantes a Falhas I

Modelos de Falha e Detecção

Primeira de três aulas sobre tolerância a falhas em sistemas distribuídos

AGENDA

O que vamos ver hoje

Aula 21 de 66h — 1ª de 3 aulas sobre tolerância a falhas.

01

Por que tolerar falhas

Falha parcial como marca registrada de sistemas distribuídos

02

Modelos de falha

Crash, omissão, timing, resposta e arbitrária (bizantina)

03

Fail-stop vs. silenciosa

O que torna uma falha fácil ou difícil de tratar

04

Detecção de falhas

Heartbeat, ping, timeout e failure detectors

05

O trade-off central

Detectar rápido x detectar com certeza

Por que tolerância a falhas é central em SD

Em um sistema centralizado, falha geralmente significa parar tudo. Em um sistema distribuído, cada processo, enlace de rede e disco é um ponto adicional de falha — e o sistema como um todo deve continuar funcionando mesmo quando parte dele já não funciona.

1

Mais componentes

Cada processo, réplica, enlace ou disco extra aumenta a probabilidade combinada de que algo, em algum lugar, esteja falhando a qualquer instante.

2

Falha parcial

O sistema pode estar parcialmente operante: alguns nós no ar, outros não — diferente de um sistema centralizado, que falha por inteiro.

3

Independência das falhas

Componentes falham de forma independente uns dos outros, o que exige que o software trate isso, não a sorte.

Modelos de falha (Cristian, 1991)

Tipo	O que acontece	Exemplo
Crash	O processo para de executar e não retoma; simplesmente some.	Servidor trava e não reinicia sozinho.
Omissão (send/receive)	O processo falha em enviar ou em receber mensagens que deveria.	Pacote é perdido na fila de saída ou de entrada.
Timing	A resposta chega, mas fora do intervalo de tempo esperado.	Vídeo chega atrasado e quebra o tempo real.
Resposta (valor / estado)	A resposta chega no tempo certo, mas com valor errado ou desvio de fluxo.	Consulta devolve o saldo errado.
Arbitrária (bizantina)	O processo produz qualquer resposta, a qualquer momento — inclusive de forma maliciosa.	Nó comprometido envia dados forjados.

Fail-stop x falhas silenciosas/bizantinas



Fail-stop

Subconjunto das falhas de crash em que os demais processos conseguem detectar, com certeza, que o processo falhou (ex.: por um mecanismo de timeout confiável).

Como a falha é limpa e detectável, é o caso mais fácil de tratar: basta reagir a um sinal claro de ausência.



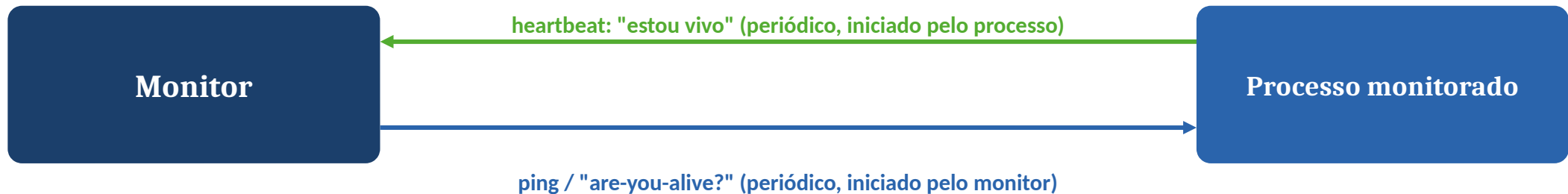
Silenciosas / bizantinas

O processo continua ativo mas se comporta de forma incorreta — devolve valores errados, desvia do algoritmo esperado ou, no limite, age de forma arbitrária/maliciosa.

É difícil distinguir 'processo lento' de 'processo morto' ou 'processo mentindo': não há um sinal limpo de falha para reagir.

Heartbeat e ping periódico

Duas estratégias simétricas para monitorar se um processo remoto ainda está ativo:



Heartbeat

O processo monitorado envia sinais periódicos por conta própria. Ausência de heartbeat por um período sugere falha; usa mais rede em regime normal, mas dá sinal ativo constante.

Ping / timeout

O monitor pergunta periodicamente e aguarda resposta dentro de um prazo. Se o prazo (timeout) expira sem resposta, o processo é declarado suspeito de falha.

Failure detectors não confiáveis (Chandra & Toueg)

Em sistemas assíncronos não existe limite superior garantido para o atraso de uma mensagem — logo, nenhum timeout consegue distinguir com 100% de certeza um processo lento de um processo morto. Chandra e Toueg (1996) formalizaram essa incerteza com o conceito de detector de falha não confiável.



O que é

Um módulo local a cada processo que mantém uma lista de processos que ele suspeita estarem falhos, com base em timeouts.



Não confiável

Pode errar: suspeitar de um processo que na verdade está ativo (falso positivo), ou demorar a suspeitar de um processo que já falhou.



Propriedades desejadas

Completude: eventualmente suspeita de todo processo que de fato falhou. Acurácia: idealmente não suspeita de processos corretos.

Rápido x certo: o trade-off da detecção

Timeout curto

- + Detecta falhas reais rapidamente, reduzindo o tempo em que o sistema opera sem saber de um componente morto.
- Mais suscetível a falsos positivos: uma rede congestionada ou um processo apenas lento é confundido com uma falha.

Timeout longo

- + Mais preciso: dá tempo suficiente para respostas legítimas atrasadas chegarem, reduzindo falsos positivos.
- Reage devagar a falhas reais: o sistema continua operando 'no escuro' por mais tempo antes de agir.

Não existe timeout perfeito em sistemas assíncronos — apenas um ponto de equilíbrio escolhido conforme o custo de cada tipo de erro.

O que levar desta aula

1 Sistemas distribuídos multiplicam os pontos de falha — tolerar falhas é requisito, não opcional.

2 A taxonomia de Cristian organiza as falhas por gravidade: crash, omissão, timing, resposta e, no limite, arbitrária/bizantina.

3 Fail-stop é o caso fácil (falha detectável com certeza); falhas silenciosas e bizantinas são o caso difícil.

4 Heartbeat, ping e timeout são os blocos básicos de detecção; failure detectors formalizam a incerteza inerente a sistemas assíncronos.

PRÓXIMA AULA

Aula 22 — Sistemas Tolerantes a Falhas II

Replicação e Redundância

Depois de mapear como um sistema falha e como detectamos essas falhas, o próximo passo é evitar que uma falha vire indisponibilidade: réplicas de dados, redundância de processos e as estratégias de consistência que sustentam essa redundância.

A 3ª aula da sequência (Aula 23) fecha o assunto com recuperação e um estudo de caso.

REFERÊNCIAS

Bibliografia desta aula

TANENBAUM, Andrew S.; VAN STEEN, Maarten.

Distributed Systems: Principles and Paradigms (ou edição atual Distributed Systems). Pearson. — Capítulo sobre tolerância a falhas: modelos de falha, fail-stop e failure detectors.

COULOURIS, George et al.

Sistemas Distribuídos: Conceitos e Projeto. 5ª ed. Porto Alegre: Bookman. — Capítulo sobre tolerância a falhas: taxonomia de falhas de Cristian e detecção.

CHANDRA, Tushar Deepak; TOUEG, Sam.

"Unreliable Failure Detectors for Reliable Distributed Systems". Journal of the ACM, 1996. — Artigo original que formaliza os detectores de falha não confiáveis.